

You Only Bark Once: Employing Object Detection to Guide a Wheeled Robot-Dog

Djayco Coret
Autonomous Systems

Tilburg University

Student ID: 123456

your.name@tilburguniversity.edu

Ivo Snels
Autonomous Systems

Tilburg University

Student ID: 123456

your.name@tilburguniversity.edu

Abstract—Placeholder

Index Terms—Edge AI, Embodiment, Autonomous Systems, Robotics

I. INTRODUCTION

II. PRACTICAL IMPLEMENTATION

The agent is organised through three layers of software abstraction: brain, robot, and sensors and actuators.

A. Sense

The agent perceives its environment through a distance sensor (sonar) and camera. The distance sensor outputs a decimal between 0 and 1, the camera outputs a [dimensions needed] RGB-image.

The data from the distance sensor is turned into a binary value, which is true iff it is safe to move.

The RGB-image gets processed by a pre-trained object detection model, yolo26n [reference needed]. The centre of the bounding box is used as x and y coordinate of the detected object. A normalised offset value is calculated using the following function f :

$$f(x, w) = \left(x - \frac{w}{2}\right) \cdot \frac{1}{w} \quad (1)$$

$$= \frac{x}{w} - \frac{1}{2} \quad (2)$$

where x is the x -coordinate of the centre of the target and w is the width of the image, which normalises the distance from the center by the width of the image; where a positive number indicates it is to the right of the center. For each object detected by the model the following features are returned: class, confidence, x , y , and offset.

B. Think

C. Act

The agent can change its environment by moving on a plane and emitting various sound through a bluetooth box.

It has two dc-motors connected to wheels, making it capable of moving along the x -axis as well as rotate along the y -axis.

For audio, it uses a subprocess to call `aplay`, that runs in a separate thread to ensure non-blocking behaviour as audio samples play.

The behaviour will be explained through its corresponding states. Consequently, the behaviour is best understood with a good grasp of the state logic. An overview follows:

a) *IDLE*: The agent will stop its motors and set its wander state to start.

b) *CHASING*: The last state change gets adjusted, such that one can keep track of how long the agent has already been chasing.

The offset for the first instance of cat gets used from the data collected at the sense step. The offset is then subtracted from the inner wheel speed, resulting in the agent to move towards the centre of the perceived bounding box.

c) *BARKING*: Plays a bark audio sample.

d) *LOST TARGET*: The agent will reduce its speed. The rational behind this is that the agent most likely loses a target because of yolo inconsistencies, or because it moves too rapidly. By using this approach, we have aimed to mitigate these target losses by this behaviour.

Alternative approaches were considered, such as spinning the agent, which have not made

e) *TARGETTING*: The agent will spin in the direction to minimise the offset. If the offset is smaller than a pre-specified number, a variable gets set to true, allowing for a state change in the next action-perception cycle.

f) *WANDERING*: The wander state is its own finite state machine. Yet these are stochastically

The agent will first check if it has been in the same state for shorter than a pre-specified value. It will return if false. Otherwise it will continue the program.

At the start of the wander state a random wander state will be chosen to be performed till the main state or the wander state changes. Wander states include: forward, backward, turn left, turn right, and spin (which is more of a tail waggle, because of a mistake made in code, but a feature worth keeping nonetheless).

g) *BLOCKED*: The agent will first check if it has been in the same state for longer than a pre-specified value. Agent will bark, update the last state change, and go backwards.

III. DISCUSSION

Intro to discussion

The motors had inconsistent speeds at similar PWM signals, resulting in the agent steering to one side. This was mitigated by using values to scale the PWM signals, yet these had to be changed as the battery depleted.

Ultimately, movement requires some proprioceptive abilities about how one has moved.

inference time

distance sensor on surfaces at an angle.

the distance sensor returns a binary value to the brain, limiting complex behaviour dependant on distance.

the reinforcement learning exploited too quickly and was not complex enough and needed

the last state change is not really a last state change and adds confusion and unnecessary complexon

IV. CONTRIBUTIONS

A. Djayco Coret

Made the motor circuit.

Managed linux environment.

Wrote abstractions for sensors and actuators.

Wrote robot python object.

Wrote vision to motion pipeline.

Wrote brain python object.

Wrote docstrings for knowledge transfer.

Wrote parts of the report.

B. Ivo Snels

REFERENCES

APPENDIX A

PICTURES

APPENDIX B

CODE

brain.py

```
1 import json
2 import random
3 from enum import Enum, auto
4 from time import time
5
6 import pandas as pd
7
8 from helper_functions import softmax
9 from robot import Robot
10
11
12 class State(Enum):
13     IDLE = auto()
14     WANDERING = auto()
15     CHASING = auto()
16     BARKING = auto()
17     BLOCKED = auto()
18     TARGETTING = auto()
19     LOST_TARGET = auto()
20
21
22 class Wander_state(Enum):
23     START = auto()
24     TURN_LEFT = auto()
25     TURN_RIGHT = auto()
```

```
FORWARD = auto()
BACKWARD = auto()
SPIN = auto()
STOP = auto()
```

```
class Brain:
```

```
    """
```

```
    A class representing the brain of the
    robot for the Autonomous Systems
    course 2026.
```

```
    The Brain class represents the controller
    part of the robot.
```

```
    The class is responsible for perception,
    state changes, and executing behaviour
    .
```

Attributes

```
    robot : Robot
```

```
        The robot (body) that the brain is
        controlling
```

```
    state : State
```

```
        The current state of the robot
```

```
    wander_state : Wander_state
```

```
        The current wander state
```

```
    wander_state_probs : list[float]
```

```
        A list of floating point decimals that
        contain logits which corresponds
        to most wandering states
```

```
    last_state_change : time.time()
```

```
        The last time when a state change was
        updated (inconsitent: does not
        update with every state change)
```

```
    learning_rate : float
```

```
        The learning rate of the brain; used
        to update the wander_state_probs
```

```
    state_duration_change : float
```

```
        How long it will take for it to stay
        in a certain state (wandering
        behaviour)
```

```
    target_in_centre : bool
```

```
        Boolean that returns if the target is
        in the centre of the frame
```

```
    """
```

```
    def __init__(self, robot, learning_rate):
```

```
        """
```

```
        Initialises the brain
```

Parameters

```
    robot : Robot
```

```
        The robot (body) that the brain is
        controlling
```

```
    learning_rate : float
```

```
        The learning rate of the brain;
        used to update the
        wander_state_probs
```

```
    """
```

```
    self.robot = robot
```

```
    self.state = State.IDLE
```

```
    self.wander_state = Wander_state.START
```

```
    self.wander_state_probs = [random.
        random() for _ in range(1, len(
        Wander_state))]
```

```

74         self.last_state_change = time()
75         self.learning_rate = learning_rate
76         self.state_duration_change = 1.5
77         self.target_in_centre = False
78
79     @classmethod
80     def from_config(cls, config_path):
81         """
82         Initialeses the brain from a
            configuration file
83
84         Parameters
85         _____
86         config_path : string
87             The path to the json file in which
            the gpio and settings are
            described
88
89         Returns
90         _____
91         cls : Brain
92             A preconfigured brain class
93         """
94         robot = Robot.from_config(config_path)
95
96         with open(config_path, "r") as file:
97             config = json.load(file)
98             learning_rate = config["learning_rate"]
99         return cls(robot, learning_rate)
100
101     @classmethod
102     def dummy_config(
103         cls,
104         dummy_img="files/test_images/cat.jpg",
105         webcam=False,
106         yolo="yolo26n.pt",
107         distance=0.2,
108         learning_rate=0.05,
109     ):
110         """
111         Initialises the brain with a dummy
            configuration, capable of running
            on non gpio devices
112
113         Parameters
114         _____
115         dummy_img : string
116             An image that will be used to
            simulate the camera of the
            brain
117         webcam: bool
118             True iff you want to use your
            webcam as camera input for the
            dummy brain
119         yolo : string
120             Path containing the yolo model
            which will be used for
            inference
121         distance : float
122             The simulated distance the
            distance sensor picks up
123
124         Returns
125         _____
126         cls : Brain
127             A dummy configured brain
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999

```

```

184         if verbose:
185             print(f"Current_State:_{self.state}")
186             print(f"Wander_State:_{self.wander_state}")
187
188         self._execute_behaviour(df)
189
190     def _perceive(self):
191         """
192         The method that is responsible for
193         capturing the external environment
194
195         Returns
196         -----
197         df : pandas.DataFrame
198             A dataframe containing the object
199             which have been captured by
200             the object detection model
201
202             * 'class' : the class of the
203                 detected object, in string
204                 format
205             * 'confidence' : the assigned
206                 probability to the detection
207             * 'x' : the x position of the
208                 centre of the detected object
209             * 'y' : the y position of the
210                 centre of the detected object
211             * 'offset' : the scaled offset
212                 from the centre of the camera
213                 frame to the centre of the
214                 detected object
215
216         safe : bool
217             True iff captured distance greater
218             than safe distance (when
219             nothing is in front)
220
221         """
222         frame = self.robot.capture_image()
223         df = self.robot.perceive(frame,
224                                 conf_threshold=0.6)
225         safe = self.robot.can_move_fwd()
226
227         return df, safe
228
229     def _state_logic(self, df, safe):
230         """
231         A method for transitioning to the
232         right states.
233
234         [full explanation here]
235
236         Parameters
237         -----
238         df : pandas.DataFrame
239             A dataframe containing the object
240             which have been captured by
241             the object detection model
242
243             * 'class' : the class of the
244                 detected object, in string
245                 format
246             * 'confidence' : the assigned
247                 probability to the detection
248             * 'x' : the x position of the
249                 centre of the detected object
250
251             * 'y' : the y position of the
252                 centre of the detected object
253             * 'offset' : the scaled offset
254                 from the centre of the camera
255                 frame to the centre of the
256                 detected object
257
258         safe : bool
259             True iff captured distance greater
260             than safe distance (when
261             nothing is in front)
262
263         """
264         found = self.class_in_frame(df, "cat")
265
266         print(found)
267         if not safe:
268             self.state = State.BLOCKED
269             return
270
271         if self.state == State.BLOCKED and
272            found:
273             self.state = State.BARKING
274
275         elif self.state == State.BLOCKED and
276            safe:
277             self.state = State.IDLE
278
279         elif self.state == State.IDLE and
280            found and not self.
281            target_in_centre:
282             self.state = State.TARGETTING
283
284         elif self.state == State.TARGETTING
285            and self.target_in_centre:
286             self.state = State.CHASING
287
288         elif (
289             self.state == State.LOST_TARGET
290             and not found
291             and time() - self.
292             state_duration_change > self.
293             last_state_change
294         ):
295             self.state = State.WANDERING
296
297         elif (
298             self.state == State.IDLE
299             and not found
300             and time() - self.
301             state_duration_change > self.
302             last_state_change
303         ):
304             self.state = State.WANDERING
305             self.last_state_change = time()
306
307         elif self.state == State.WANDERING and
308            found:
309             self.state = State.IDLE
310             self.last_state_change = time()
311
312         elif self.state == State.IDLE and
313            found:
314             wander_states = list(Wander_state)
315             if self.wander_state !=
316                Wander_state.START:
317                 state_in_which_found = (
318                     wander_states.index(self.
319                     wander_state) - 1

```

```

275         ) # exclude start state
276         self.wander_state_probs[
277             state_in_which_found] +=
278             self.learning_rate
279         self.state = State.CHASING
280         self.last_state_change = time()
281
282     elif self.state == State.CHASING and
283         not found:
284         self.state = State.LOST_TARGET
285
286     elif found:
287         self.offset = self.class_offset(df,
288             "cat")
289         print(self.offset)
290         self.state = State.CHASING
291
292     elif self.state == State.CHASING and
293         not found:
294         self.state = State.WANDERING
295
296 def _execute_behaviour(self, df):
297     """
298     Method that executes the behaviour
299     based on the current state (state
300     is an implicit parameter)
301
302     Parameters
303     -----
304     df : pandas.DataFrame
305         A dataframe containing the object
306         which have been captured by
307         the object detection model
308
309     * 'class' : the class of the
310         detected object, in string
311         format
312     * 'confidence' : the assigned
313         probability to the detection
314     * 'x' : the x position of the
315         centre of the detected object
316     * 'y' : the y position of the
317         centre of the detected object
318     * 'offset' : the scaled offset
319         from the centre of the camera
320         frame to the centre of the
321         detected object
322
323     """
324     if self.state == State.IDLE:
325         self.robot.stop()
326         self.wander_state = Wander_state.
327             START
328
329     elif self.state == State.CHASING:
330         self.last_state_change = time()
331         offset = self.class_offset(df, "
332             cat")
333         self.robot.chase(offset)
334
335     elif self.state == State.BARKING:
336         self.robot.bark()
337
338     elif self.state == State.LOST_TARGET:
339         left_speed, right_speed = self.
340             robot.return_motor_speeds()
341         left_speed = left_speed - 0.05
342         right_speed = right_speed - 0.05
343
344         self.robot.forward([left_speed,
345             right_speed])
346
347     elif self.state == State.TARGETTING:
348         offset = self.class_offset(df, "
349             cat")
350         if offset < 0:
351             self.robot.spin("L", 0.7)
352         else:
353             self.robot.spin("R", 0.7)
354
355         if abs(offset) < 0.05:
356             self.target_in_centre = True
357         else:
358             self.target_in_centre = False
359
360     elif self.state == State.WANDERING:
361         if time() - self.last_state_change
362             > self.state_duration_change:
363             self.state = State.IDLE
364             self.last_state_change = time
365             ()
366             return
367
368         if self.wander_state ==
369             Wander_state.START:
370             possible_states = list(
371                 Wander_state)[1:] #
372                 excludes start state
373             self.wander_state = random.
374                 choices(
375                     possible_states, weights=
376                         softmax(self.
377                             wander_state_probs)
378                     ) [0]
379             self.last_state_change = time
380             ()
381
382         if self.wander_state ==
383             Wander_state.TURN_LEFT:
384             self.robot.turn("L", 0.85)
385         elif self.wander_state ==
386             Wander_state.TURN_RIGHT:
387             self.robot.turn("R", 0.85)
388         elif self.wander_state ==
389             Wander_state.FORWARD:
390             self.robot.forward(0.85)
391         elif self.wander_state ==
392             Wander_state.BACKWARD:
393             self.robot.backward(0.85)
394         elif self.wander_state ==
395             Wander_state.SPIN:
396             right = int(random.random() >
397                 0.5)
398             direction = ["L", "R"][right]
399             self.robot.spin(direction,
400                 0.4)
401         elif self.wander_state ==
402             Wander_state.STOP:
403             self.robot.stop()
404
405     elif self.state == State.BLOCKED:
406         if time() - self.last_state_change
407             > self.state_duration_change:
408             self.robot.bark()
409             self.last_state_change = time
410             ()

```

```

368         self.robot.backward(0.6)
369
370     def class_in_frame(self, df, class_: str):
371         """
372         A method that return iff there is an
373         object of a prespecified class in
374         the dataframe.
375
376         Parameters
377         -----
378         df : pandas.DataFrame
379             A dataframe containing the object
380             which have been captured by
381             the object detection model
382
383             * 'class' : the class of the
384                 detected object, in string
385                 format
386             * 'confidence' : the assigned
387                 probability to the detection
388             * 'x' : the x position of the
389                 centre of the detected object
390             * 'y' : the y position of the
391                 centre of the detected object
392             * 'offset' : the scaled offset
393                 from the centre of the camera
394                 frame to the centre of the
395                 detected object
396         class_ : str
397             The class which will be detected
398
399         Returns
400         -----
401         value : bool
402             True iff there is a class_ in the
403             dataframe
404
405         """
406         value = df["class"].str.contains(
407             class_).any()
408
409         return value
410
411     def class_offset(self, df, class_: str):
412         """
413         A method that returns the offset of
414         the first object of a prespecified
415         class in the dataframe.
416
417         Parameters
418         -----
419         df : pandas.DataFrame
420             A dataframe containing the object
421             which have been captured by
422             the object detection model
423
424             * 'class' : the class of the
425                 detected object, in string
426                 format
427             * 'confidence' : the assigned
428                 probability to the detection
429             * 'x' : the x position of the
430                 centre of the detected object
431             * 'y' : the y position of the
432                 centre of the detected object
433             * 'offset' : the scaled offset
434                 from the centre of the camera

```

```

435         frame to the centre of the
436         detected object
437         class_ : str
438             The class which will be detected
439
440         Returns
441         -----
442         offset : float
443             The offset of the first row
444             containing that class
445
446         """
447         if self.class_in_frame(df, class_):
448             return float(df[df["class"] ==
449                             class_].iloc[0]["offset"])
450         else:
451             return 0
452
453 if __name__ == "__main__":
454     brain = Brain.from_config("gpio_settings.
455                               json")
456     # brain = Brain.dummy_config(webcam=False)
457
458     try:
459         while True:
460             brain.update(verbose=True)
461
462     except KeyboardInterrupt:
463         brain.stop()

```

```

464
465 robot.py
466
467 import json
468 import time
469 from time import sleep
470
471 from dummy_robot import (
472     Dummy_audio,
473     Dummy_camera,
474     Dummy_distance_sensor,
475     Dummy_motor,
476     Webcam,
477 )
478 from helper_functions import get_wav_files
479 from visual_processing import
480     Visual_processing
481
482 class Robot:
483     """A class representing a robot for the
484         autonomous systems course 2026
485
486     Attributes
487     -----
488     left motor : Motor
489         The left motor of the robot, oriented
490         with the raspberry facing forwards
491     right motor : Motor
492         The right motor of the robot, oriented
493         with the raspberry facing
494         forwards
495     audio : Audio_processing
496         The audio interface of the robot
497     visual_proc : Visual_processing
498         The object detection pipeline
499     camera : Camera
500         The camera sensor of the robot

```

```

31 safety_distance : float
32     The minimum distance between the robot
33     and the object detected by the
34     distance sensor
35 verbose : bool
36     True if you want stuff printed in the
37     console
38 scalar_left: float
39     a scalar used to scale the max of the
40     left wheel
41 scalar_right: float
42     a scalar used to scale the max of the
43     right wheel
44 """
45
46 def __init__(
47     self,
48     left_motor,
49     right_motor,
50     audio_proc,
51     vis_proc,
52     distance_sensor,
53     camera,
54     verbose=False,
55 ):
56     """Initialises the robot
57
58     Parameters
59     -----
60     left motor : Wheel
61         The left motor of the robot,
62         oriented with the raspberry
63         facing forwards
64     right motor : Wheel
65         The right motor of the robot,
66         oriented with the raspberry
67         facing forwards
68     audio : Audio_processing
69         The audio interface of the robot.
70     visual_processing : Visual_processing
71         The visual processing part of the
72         robot.
73     distance_sensor : Distance_sensor
74         The depth measuring capabilities
75         of the robot.
76     camera : Camera
77         The camera of the robot.
78     verbose: bool
79         Verbosity
80     """
81     self.left_motor = left_motor
82     self.right_motor = right_motor
83     self.audio = audio_proc
84     self.visual_proc = vis_proc
85     self.distance_sensor = distance_sensor
86     self.camera = camera
87
88     self.safety_distance = 0.3
89     self.verbose = verbose
90
91     self.scalar_left = 0.85
92     self.scalar_right = 1
93
94 @classmethod
95 def from_config(cls, json_path):
96     """
97     Initialises the robot from a config

```

```

98     file
99
100 Parameters
101 -----
102 json_path: str
103     The file path of the json file (e.
104     g., 'gpio_settings.json')
105 """
106 # imports inside the config because
107 # otherwise it does not run on
108 # laptops
109 from camera import Camera
110 from sensors_actuator_wrappers import
111     Audio_processing, Distance_sensor,
112     Motor
113
114 with open(json_path, "r") as file:
115     gpio_settings = json.load(file)
116
117 right_motor = Motor(
118     gpio_settings["forward_pin_right"]
119     ],
120     gpio_settings["backward_pin_right"]
121     ],
122     gpio_settings["enable_pin_right"],
123 )
124
125 left_motor = Motor(
126     gpio_settings["forward_pin_left"],
127     gpio_settings["backward_pin_left"]
128     ],
129     gpio_settings["enable_pin_left"],
130 )
131
132 distance_sensor = Distance_sensor(
133     gpio_settings["
134         distance_trigger_pin"],
135     gpio_settings["
136         distance_echo_pin"]
137 )
138
139 barks = get_wav_files(gpio_settings["
140     bark_dir"])
141 growls = get_wav_files(gpio_settings["
142     growl_dir"])
143 audio = Audio_processing(barks, growls
144 )
145
146 confidence_threshold = gpio_settings["
147     confidence_threshold"]
148 visual_proc = Visual_processing(
149     gpio_settings["yolo_model"],
150     confidence_threshold
151 )
152
153 camera = Camera()
154
155 return cls(left_motor, right_motor,
156     audio, visual_proc,
157     distance_sensor, camera)
158
159 @classmethod
160 def dummy_config(cls, dummy_image,
161     yolo_model, webcam=False, distance
162     =0.5):
163     """Initialises the robot with dummy
164     hardware abilities, giving one the

```

```

        capacity to run on device other than raspberry pi.
132
133 Parameters
134
135 dummy_image: str
136     the path of the image used as
137     input for the fake 'camera'
138     note: will not be used when using
139     webcam
140 yolo_model: str
141     the path of the yolo model that
142     will be used
143 webcam: bool
144     True if you want to use your own
145     webcam
146
147 """
148 right_motor = Dummy_motor("right")
149 left_motor = Dummy_motor("left")
150
151 distance_sensor =
152     Dummy_distance_sensor(distance)
153
154 audio = Dummy_audio()
155
156 visual_proc = Visual_processing(
157     yolo_model)
158
159 if webcam:
160     camera = Webcam()
161 else:
162     camera = Dummy_camera(dummy_image)
163
164 verbose = True
165
166 return cls(
167     left_motor,
168     right_motor,
169     audio,
170     visual_proc,
171     distance_sensor,
172     camera,
173     verbose,
174 )
175
176 def return_motors(self) -> list:
177     """
178     Return the wheels of the robot in a
179     list
180     """
181     return [self.left_motor, self.
182             right_motor]
183
184 def return_motor_speeds(self) -> tuple:
185     """
186     Returns a tuple of the motor speeds
187
188     Returns
189     """
190     speeds : tuple[float]
191     The tuple of speeds
192     """
193     return tuple(motor.speed for motor in
194                  self.return_motors())
195
196 def return_motor_scalars(self):
197     """

```

Returns a list of the scalars by which the speed of the motors is scaled by

Returns

```

scalars = list[float]
    The list of scalars.
"""

```

```

scalar = [self.scalar_left, self.
          scalar_right]
return scalar

```

```
def forward(self, speed=[1.0, 1.0]):
```

```

"""
Changes the state of the robot to
going forward

```

Parameters

```

speed : float [0, 1] or list[float]
    The speed at which the agent will
    go forward
"""

```

```

if isinstance(speed, float): #
    support for when you do only one
    number
    speed = [speed for _ in range(len(
        self.return_motors()))]

```

```

for i, motor in enumerate(self.
    return_motors()):
    motor.forward(speed[i] * self.
        return_motor_scalars()[i])

```

```
def stop(self, stop_cam=False):
```

```

"""
Changes the state of the robot to
stopped
"""

```

```

for motor in self.return_motors():
    motor.stop()

```

```

if stop_cam:
    self.camera.stop()

```

```
def backward(self, speed=1):
```

```

"""
Changes the state of the robot to
going backward

```

Parameters

```

speed : float [0 , 1]
    The speed at which the agent will
    go backward
"""

```

```

for motor, scalar in zip(self.
    return_motors(), self.
    return_motor_scalars()):
    motor.backward(speed * scalar)

```

```
def turn(self, direction: str, speed=1.0):
```

```

"""
Makes the robot turn

```



```

240     Stops one wheel, while making the
241         other go forward.
242
243     Parameters
244     -----
245     direction : str ('L', 'R')
246         The direction how the robot will
247         spin
248     speed : float
249         The speed of the motors
250     """
251     if direction.upper() == "R":
252         self.left_motor.forward(speed *
253             self.return_motor_scalars()
254             [0])
255         self.right_motor.stop()
256     elif direction.upper() == "L":
257         self.right_motor.forward(speed *
258             self.return_motor_scalars()
259             [1])
260         self.left_motor.stop()
261     else:
262         raise ValueError("Direction_must_
263             be_either_left_or_right")
264
265 def spin(self, direction, speed):
266     """
267     Makes the robot spin
268
269     Makes wheels turn in opposite
270     direction
271
272     Parameters
273     -----
274     direction : str ('L', 'R')
275         The direction how the robot will
276         spin
277     speed : float
278         The speed of the motors
279     """
280     if direction.upper() == "R":
281         self.left_motor.forward(speed *
282             self.return_motor_scalars()
283             [0])
284         self.right_motor.backward(speed *
285             self.return_motor_scalars()
286             [1])
287     elif direction.upper() == "L":
288         self.right_motor.forward(speed *
289             self.return_motor_scalars()
290             [1])
291         self.left_motor.backward(speed *
292             self.return_motor_scalars()
293             [0])
294     else:
295         raise ValueError("Direction_must_
296             be_either_left_('L')_or_right_
297             ('R')")
298
299 def bark(self):
300     """
301     Plays a randomly selected bark sample
302     """
303     self.audio.bark()
304
305 def growl(self):
306     """

```

```

288     Plays a randomly selected growl sample
289     """
290     self.audio.growl()
291
292 def perceive(self, frame, conf_threshold
293     =0.8):
294     """
295     A method that returns a dataframe of
296     all detected object in the frame,
297     with a confidence higher than the
298     threshold.
299
300     Parameters
301     -----
302     frame : numpy.ndarray
303         The image which will be used to
304         detect
305     conf_threshold : float
306         Everything below this number will
307         not be returns
308
309     Returns
310     -----
311     df_threshold : pandas.DataFrame
312         A dataframe containing only the
313         rows which have a higher
314         confidence than the threshold
315
316         * 'class' : the class of the
317             detected object, in string
318             format
319         * 'confidence' : the assigned
320             probability to the detection
321         * 'x' : the x position of the
322             centre of the detected object
323         * 'y' : the y position of the
324             centre of the detected object
325         * 'offset' : the scaled offset
326             from the centre of the camera
327             frame to the centre of the
328             detected object
329     """
330     df = self.visual_proc.perceive(frame)
331     df_threshold = df[df["confidence"] >=
332         conf_threshold]
333     return df_threshold
334
335 def capture_image(self):
336     """Method to capture image from the
337         camera
338
339     Returns
340     -----
341     frame: NDarray
342         the image captured by the camera
343     """
344     frame = self.camera.capture()
345     return frame
346
347 def can_move_fwd(self):
348     """Method that checks if it is safe to
349         move forward
350
351     Returns
352     -----
353     safe: bool
354         True if can move forward; False if

```

```

335         it cannot.
336     """
337     safe = self.distance_sensor.
338         safe_distance(self.safety_distance
339     )
340     return safe
341
342 def chase(self, offset):
343     """Method for chasing
344
345     Sets the wheels to forwards and
346     calculates speed for the wheels to
347     steer
348
349     Parameters
350     offset : float
351         the offset from center [-0.5, 0.5]
352     """
353     print(offset)
354
355     if offset > 0:
356         self.left_motor.forward(1 * self.
357             return_motor_scalars()[0])
358         self.right_motor.forward((1 -
359             offset) * self.
360             return_motor_scalars()[1])
361     else:
362         self.left_motor.forward((1 +
363             offset) * self.
364             return_motor_scalars()[0])
365         self.right_motor.forward(1 * self.
366             return_motor_scalars()[1])
367
368 if __name__ == "__main__":
369     bot = Robot.from_config("gpio_settings.
370         json")
371
372     try:
373         while True:
374             bot.forward()
375     except:
376         bot.stop(True)

```

```

16     )
17
18 def capture(self, file_name=None):
19     frame = self.cam.capture_array()
20
21     frame_rotate = cv2.flip(frame, -1)
22
23     if file_name:
24         cv2.imwrite(f"{file_name}.jpg",
25             frame_rotate)
26
27     return frame_rotate
28
29 def stop(self):
30     self.cam.stop()
31
32 if __name__ == "__main__":
33     c = Camera()
34
35     c.capture("test")

```

```

1 import cv2
2 import pandas as pd
3 from ultralytics import YOLO
4
5 class Visual_processing:
6     """
7     A class representing the visual processing
8
9     Attributes
10
11     model : ultralytics.models.yolo.model.YOLO
12         The model that will classify the
13         images.
14     confidence_threshold : float [0, 1]
15         The threshold that will have to for
16         the system to act on the
17         classification.
18
19     """
20
21     def __init__(self, model,
22         confidence_threshold=0.8):
23         """
24         Initialises the visual processing
25         class
26
27     Parameters
28
29     model : string
30         The path of the model that will
31         classify the images.
32     confidence_threshold : float [0, 1]
33         The threshold that will have to
34         for the system to act on the
35         classification.
36
37     """
38     self.model = YOLO(model)
39     self.confidence_threshold =
40         confidence_threshold
41
42     def perceive(self, input):
43         """R
44         uns YOLO and outputs dataframe of
45         found classes.

```

```

1 camera.py
2 import cv2
3 from picamera2 import Picamera2
4
5 class Camera:
6     def __init__(self) -> None:
7         self.cam = Picamera2()
8         config = self.cam.
9             create_still_configuration(
10                 main={"format": "RGB888", "size":
11                     (1280, 720)}
12             )
13         self.cam.configure(config)
14         self.cam.start()
15
16         self.cam.set_controls(
17             {"ExposureTime": 2000, "
18                 AnalogueGain": 20.0, "AeEnable
19                 ": False}

```

```

Parameters
_____
input : np.array
    The captured image as a numpy
    array

Returns
_____
df : pandas.DataFrame
    A dataframe containing the object
    which have been captured by
    the object detection model

    * 'class' : the class of the
      detected object, in string
      format
    * 'confidence' : the assigned
      probability to the detection
    * 'x' : the x position of the
      centre of the detected object
    * 'y' : the y position of the
      centre of the detected object
    * 'offset' : the scaled offset
      from the centre of the camera
      frame to the centre of the
      detected object

"""
_, w_img, _ = input.shape

results = self.model.predict(input)

frame = list()

for result in results:
    boxes = result.boxes
    for box in boxes:
        row = dict()
        cls_id = int(box.cls[0])
        cls = result.names[cls_id]
        x, y, _, _ = box.xywh[0]
        row["class"] = result.names[
            cls_id]
        row["confidence"] = float(box.
            conf[0])
        row["x"], row["y"] = float(x),
            float(y)
        row["offset"] = float(x) /
            w_img - 0.5

        frame.append(row)

    if box.conf[0] >= self.
        confidence_threshold:
        print(f"{cls}_at_{(x),_y
            }),_with_confidence_{
            box.conf[0]}")

df = pd.DataFrame(frame, columns=["
    class", "confidence", "x", "y", "
    offset"])

return df

```

```

from random import randint

```

```

from gpiozero import DigitalOutputDevice,
    DistanceSensor, PWMOutputDevice

from helper_functions import check_speed,
    play_wav

class Audio_processing:
    """A class representing the audio making
    module of the robot

    Attributes
    _____
    barks : List[str]
        A list containing the path at which
        the various audio samples of barks
        reside
    growl : List[str]
        A list containing the path at which
        the various audio samples of
        growls reside"""

    def __init__(self, bark, growl):
        """Initialises the Audio_processing
        class

        Parameters
        _____
        bark : List(str)
            A list containing one or several
            paths of a bark audio sample
            in wav format
        growl : List(str)
            A list containing one or several
            paths of a growl audio sample
            in wav format

        """
        self.bark_ = list()
        self.bark_.extend(bark)

        self.growl_ = list()
        self.growl_.extend(growl)

    def bark(self):
        """Plays a random bark audio sample"""
        max_len = len(self.bark_)
        index = randint(0, max_len - 1)
        path = self.bark_[index]
        play_wav(path)

    def growl(self):
        """Plays a random growl audio sample
        """
        max_len = len(self.growl_)
        index = randint(0, max_len - 1)
        path = self.growl_[index]
        play_wav(path)

class Distance_sensor:
    def __init__(self, trigger_pin, echo_pin):
        self.sensor = DistanceSensor(
            trigger_pin, echo_pin)

    def safe_distance(self, safe_distance:
        float):

```

```

54         print(self.sensor.distance)
55         return self.sensor.distance >
            safe_distance
56
57
58 class Motor:
59     """A class representing a motor
60
61     Attributes
62     _____
63     PWM : PWMOutputDevice
64         The pin number connected to the Pulse
        Width Modulation connection on the
        L293D chip.
65         This pin changes the speed of the
        motor.
66     forward_pin : DigitalOutputDevice
67         The pin number connected to the
        forward connection on the L293D
        chip
68     backward_pin : DigitalOutputDevice
69         The pin number connected to the
        backward connection on the L293D
        chip
70
71     """
72     def __init__(self, forward_pin,
73                 backward_pin, pwm_pin):
74         """Initialise the motor class
75
76         Parameters
77         _____
78         pwm_pin : int
79             The pin connected to the Pulse
            Width Modulation connection on
            the L293D chip.
80             This pin changes the speed of the
            motor.
81         forward_pin : int
82             The pin connected to the forward
            connection on the L293D chip
83         backward_pin : int
84             The pin connected to the backward
            connection on the L293D chip
85         speed : float
86             current speed
87
88         """
89         self.PWM = PWMOutputDevice(pwm_pin)
90         self.forward_pin = DigitalOutputDevice(
91             forward_pin)
92         self.backward_pin =
93             DigitalOutputDevice(backward_pin)
94         self.speed = 0
95
96     def __repr__(self):
97         return f"Motor_object:_forward_pin:_{
98             self.forward_pin},_backward_pin:_{
99                 self.backward_pin},_PWM:_{self.PWM
100                 },_speed:_{self.PWM.value}"
101
102     def forward(self, speed=1.0):
103         """Makes the motor move in the
104             direction such that the agent
105             moves forward
106
107         Parameters
108         _____
109
110         speed : float
111             The input speed

```

```

100     speed : float [0, 1]
101         The speed at which the motor moves
102     """
103     self.speed = check_speed(speed)
104     self.PWM.value = self.speed
105     self.backward_pin.off()
106     self.forward_pin.on()
107
108     def stop(self):
109         """Makes the motor stop"""
110         self.speed = 0
111         self.backward_pin.off()
112         self.forward_pin.off()
113
114     def backward(self, speed=1):
115         """Makes the motor move in the
116             direction such that the agent
117             moves backward
118
119         Parameters
120         _____
121         speed : float [0, 1]
122             The speed at which the motor moves
123     """
124     self.speed = check_speed(speed)
125     self.PWM.value = self.speed
126     self.forward_pin.off()
127     self.backward_pin.on()

```

```

1 import subprocess
2 from pathlib import Path
3
4 import numpy as np
5
6 def softmax(scores):
7     """
8     A function that returns the input list as
9     a probability distribution (softmax).
10
11     Parameters
12     _____
13     scores : list[float]
14         list that will be turned into a pdf
15
16     Returns
17     _____
18     pd : list[float]
19         The probability distribution
20     """
21     x = np.array(scores)
22     e_x = np.exp(x - np.max(x))
23     return (e_x / e_x.sum()).tolist()
24
25 def check_speed(speed):
26     """
27     A function that return a value between 0
28     and 1, which has predictable behaviour
29     for the motor class
30
31     Parameters
32     _____
33     speed : float
34         The input speed

```

```

35
36     Returns
37     -----
38     float
39         The output speed, which is limited to
40         a range of [0, 1]
41     """
42     return max(0, min(1, speed))
43
44
45 def play_wav(path):
46     """
47     A function that plays the .wav file of the
48     specified path
49
50     Parameters
51     -----
52     path : string
53         The path of the .wav file which is
54         supposed to be played
55     """
56     subprocess.run(["aplay", path])
57
58 def get_wav_files(directory_path):
59     """
60     A function that returns all the .wav files
61     in a certain directory
62
63     Parameters
64     -----
65     directory_path : string
66         The directory where the .wav files are
67
68     Returns
69     -----
70     wav_files : list[string]
71         A list containing all the .wav files
72         in the directory
73     """
74     path = Path(directory_path)
75     wav_files = [str(file) for file in path.
76                  glob("*.wav")]
77     return wav_files

```

APPENDIX C

CIRCUIT SCHEMATICS